

WiFi CSI RADAR

Uctan Uca Surec Raporu

Veri toplamadan canli olasiliksal konum tahminine — her adimin teknik aciklamasi

GENEL BORU HATTI (PIPELINE)



BU RAPOR HANGİ SORUYU CEVAPLIYOR?

"Zone A ya da B'de dururken, modelin gordugu CSI verisi onceki test serileriyle birebir ortusmuyor — yine de A mi B mi oldugunu nasil soyluyor?"

Cevap: Model BİREBİR EŞLEŞTİRME yapmaz. Egitim sirasinda binlerce ornekten GENELLENEBİLİR bir istatistiksel imza / karar siniri ogrenir; yeni gelen her ornegi bu sinira gore SINIFLANDIRIR ve olasilik uretir. Bu raporun her bolumu, bu ogrenmenin hangi veriden ve hangi islem adimlarindan gectigini en kucuk detayina kadar anlatir.

01 DONANIM — FİZİKSEL KURULUM VE PORT EŞLEMESİ

core/esp32_serial_reader.py · core/esp32_multi_reader.py

1.1 Fiziksel kurulum

- 3 adet ESP32, fiziksel olarak fan (yelpaze) diziliminde yerleştirilmiştir:
RX-1 SOL (-45°) · RX-2 ORTA (0°) · RX-3 SAĞ (+45°) — her biri 3dBi anten
- Her ESP32 USB-seri ile bilgisayara bağlıdır (ag/IP üzerinden 'ping'
ATILAMAZ — sağlık kontrolü için check_esp_connections.py yazıldı: portlari
acar, CSI akiyor mu diye bakar, '✓ CANLI / ✗ SORUNLU' raporlar)
- Her CSI paketi 64 alt-tasiyici (subcarrier) genlik degeri tasir
(N_SUBCARRIERS = 64)

1.2 Port ↔ fiziksel pozisyon eşlemesi (kalibrasyonun ilk adimi)

Sorun: find_esp32_ports() portlari ALFABETİK sirada donduruyor
(usbserial-0001, usbserial-5, usbserial-6) — bu sirada FİZİKSEL yerlesimle
(SOL/ORTA/SAĞ) ÖRTÜŞMÜYOR. Yanlis eslestirme = yanlis kalibrasyon.

Çözüm — PORT_POSITION sözlüğü (sabit fiziksel-port eslemesi):

usbserial-5	->	SOL (-45°)	position 0
usbserial-6	->	ORTA (0°)	position 1
usbserial-0001	->	SAĞ (+45°)	position 2

order_ports_by_position() bulunan portlari bu eslemeyle [SOL,ORTA,SAĞ]
fiziksel sirasina yeniden dizer; bilinmeyen bir port adi gelirse None
doner (yanlis eslemeyle devam etmesini engeller).

2.1 CSI hız artırma (traffic boosting)

ESP32'lerin doğal CSI üretim hızı çok düşük ($\sim 0.2-0.7$ Hz) — zone'ları ayırt edecek kadar veri toplamak için yetersiz.

Çözüm — TrafficBooster (UDP trafik 'sel'i):

- 1) discover_esp32_ips(): ESP32'ler DTR/RTS seri sinyalleriyle resetlenir, boot loglarından 'IP: x.x.x.x' regex ile okunur. (ICMP ping değil — macOS'te düşük aralıklarla ping atmak root ister, UDP istemez.)
Bulunan IP'ler: SOL=192.168.1.121 ORTA=192.168.1.122 SAĞ=192.168.1.120
- 2) Bu IP'lere saniyede ~ 100 adet 32-byte UDP paketi gönderilir -> 802.11 çerçeve trafiği oluşur -> ESP32'nin CSI callback'i çok daha sık tetiklenir

Sonuç: CSI hızı ~ 0.5 Hz'den $\sim 40-80$ Hz/anten'e çıkar (3 anten toplam $\sim 120-240$ ham paket/sn).

2.2 Etiketli kayıt akışı ve tespit edilen kritik sorun

```
collect_data_multi.py --label zone_A --seconds 90
```

Akis: portları bul/sirala -> IP'leri keşfet -> trafik artırmayı başlat -> ESP32MultiReader'i başlat -> 90 sn boyunca her birleşik örneği data/csi_fingerprints_multi.csv dosyasına yaz (192 sayısal sütun + label).

⚠ İlk toplamada combine_hz=20 ile sabit-hızlı ('en son neyse onu al') birleştirme kullanıldı — hızlı kanalların verisi cope gidiyor, yavaş kanaldan aynı bayat örnek tekrar tekrar kullanılıyordu (kalibrasyonsuz + israfli). O veri yedeklendi (_old_uncalibrated_*.csv) ve YENİ olay-tabanlı yöntemle (Bölüm 3) yeniden toplanıyor.

3.1 Neden 'birleştirme' bir kalibrasyon problemi midir?

Her kanal kendi arka plan thread'inde okunur ve FARKLI hızlarda akar (örn. SOL 82 Hz, ORTA 38 Hz, SAĞ 65 Hz). Tek bir 'birleşik örnek' üretmek için 3 kanaldan gelen verileri zamansal olarak hizalamak (kalibre etmek) gerekir.

Yanlış yaklaşım ('sabit aralıklarla en sonu al' / combine_hz):

- yavaş kanaldan AYNI bayat örnek defalarca tekrar kullanılır (kalibrasyonsuz)
- hızlı kanalların ürettiği örneklerin çoğu hiç kullanılmadan kaybolur (israf)

3.2 Çözüm — OLAY TABANLI / KAYIPsiz BİRLEŞTİRME (yeni yöntem)

Her kanaldan yeni paket geldiğinde o kanal threading.Lock ile 'TAZE' işaretlenir: `self._latest[idx]=arr ; self._fresh[idx]=True`

Üç kanaldan da en az bir taze örnek biriktiği an:

```
if all(self._fresh) and all(x is not None for x in self._latest):
    emit_snapshot = list(self._latest)    # ANINDA birleştir, gönder
    self._fresh = [False]*3              # bayrakları sıfırla
```

Üç garanti birden sağlanır:

- ✓ Hiçbir kanaldan veri cope gitmez (her biri en az 1 kez katkı sağlar)
- ✓ Çıkış hızı EN YAVAŞ kanalın hızına eşitlenir (gerçek kalibrasyon)
- ✓ Üç örnek de zamanda birbirine yakın anlarda toplanır

Ölçüm: `last_skew_s` (kanal-arası en büyük zaman farkı) ~10 ms; çıkış hızı eski yöntemde ~19 örnek/sn -> yeni yöntemde ~47 örnek/sn.

02 BİRLEŞİK CSI VEKTÖRÜ VE DİSKTEKİ SON HALİ

ESP32MultiReader._emit() → data/csi_fingerprints_multi.csv

4.1 192-boyutlu birleşik vektör nasıl oluşur?

_emit(): her kanaldan gelen 64'lük diziler [SOL | ORTA | SAĞ] sırasıyla uç uca eklenir (np.concatenate) -> 192 elemanlı tek 'birleşik CSI vektörü'. combined_count sayacı ve last_skew_s (kanallar arası en büyük zaman farkı) tanı amaçlı tutulur — kalibrasyon kalitesi her an ölçülebilir hale gelir.

4.2 Diskteki son hali — bir sonraki tüm adımların tek girdisi

[SOL kanal: 64 değer][ORTA kanal: 64 değer][SAĞ kanal: 64 değer]
amp_ch0_0..63 amp_ch1_0..63 amp_ch2_0..63



CSV satırı: amp_ch0_0, ..., amp_ch2_63, label
(data/csi_fingerprints_multi.csv)

Bu CSV, Bölüm 5'teki özellik mühendisliği ve Bölüm 6'daki model eğitiminin TEK girdisidir — yani kalibrasyon kalitesi, modelin öğrenebileceği 'gerçeğin' ne kadar doğru temsil edildiğini doğrudan belirler.
(örnek dağılım: zone_A=1693, zone_B=1698, empty=1793 satır)

5.1 Neden 'ham veri' değil 'öznitelik vektörü'?

Ham 192 değer (3x64 genlik) doğrudan modele verilmez — gürültüye duyarlı, boyutu yüksek ve 'şekil' bilgisini açıkça taşımaz. Onun yerine her paketten anlamlı, düşük-boyutlu bir 'parmak izi' (öznitelik vektörü) çıkarılır. Bu vektör HEM eğitimde HEM canlı tahminde AYNI fonksiyonla üretilir — (build_multi_feature_vector) bu tutarlılık, modelin öğrendiği şeyin canlı veride de geçerli olmasının ön koşuludur.

5.2 Tek-kanal işlem hattı — temizlik ve normalizasyon

Adım 1 — Ölü/sabit subcarrier tespiti find_dead_subcarriers(var_threshold=1e-3): Eğitim setinde varyansı ~0 olan alt-taşıyıcılar (pilot/DC/guard sinyalleri — bilgi taşımazlar) bulunur ve sonraki tüm adımlarda atılır.

Adım 2 — Per-packet normalizasyon normalize_row(): row / mean(sıfır-olmayan değerler) -> her paketin ortalaması 1'e çekilir. Sonuç: mutlak güç / mesafe değişimlerine karşı dayanıklılık — sadece sinyalin alt-taşıyıcılar arasındaki GÖRELİ DAĞILIM ŞEKLİ kalır (kişinin antene olan mutlak mesafesi değil, CSI'nin 'şekli' konum bilgisini taşır).

6.1 Tek-kanal 'şekil parmak izi' (her anten için ayrı üretilir)

Temizlenmiş + normalize edilmiş 'şekilden' 23 sayısal özet çıkarılır:

- 8 grup ortalaması : diziyi 8 eşit parçaya bölüp her parçanın ortalaması (spektrumun kaba şekli)
 - 7 grup farkı : $\text{np.diff}(\text{grup_ortalamları})$ — komşu gruplar arası eğim / monotonluk bilgisi
 - 1 eğim (slope) : $\text{np.polyfit derece-1} \rightarrow$ spektral 'tilt'
 - 1 ağırlık merkezi : $\text{centroid} = \Sigma(x \cdot \text{değer}) / \Sigma(\text{değer})$
 - 1 düzlük : $\text{std/mean} \rightarrow$ spektrumun ne kadar 'pürüzlü' olduğu
 - 1 log-enerji : $\log(1+|\text{ham_değer}| \cdot \text{ortalama}) \rightarrow$ mutlak güç özeti
 - 4 min/max/p25/p75 : normalize dağılımın sınırı ve çeyreklikleri
- 23 ÖZELLİK / kanal × 3 kanal = 69

6.2 YENİ — Çapraz-kanal 'yön/çeşitlilik' özellikleri (3-anten avantajı)

Sadece 3 kanalın 23'er özelliğini yan yana koymak ($3 \times 23 = 69$) antenler ARASINDAKİ farkı modele AÇIKÇA sunmaz — örtük yön/AoA bilgisini model kendi keşfetmek zorunda kalır (3-anten sonuçlarının beklenenden az üstün çıkmasının olası nedeni budur). `cross_channel_features()` bunu 6 ek özellik ile giderir:

- 3 log-enerji farkı : (SOL-ORTA), (ORTA-SAĞ), (SOL-SAĞ)
→ kişi hangi yöne yakınsa o anten enerjisi diğerlerinden farklılaşır (basitleştirilmiş açı-of-geliş / AoA bilgisi)
- 3 şekil korelasyonu : normalize şekiller arası Pearson korelasyonu
→ yansıma desenlerinin ne kadar benzer/farklı olduğu

`build_multi_feature_vector() = [kanal0(23) | kanal1(23) | kanal2(23) | çapraz(6)]`
———— TOPLAM: 75 BOYUTLU ÖZELLİK VEKTÖRÜ

7.1 Veri yükleme ve adil A/B karşılaştırma tasarımı

data/csi_fingerprints_multi.csv okunur, her satırdan 75-boyutlu özellik vektörü çıkarılır (Bölüm 6). Adil A/B karşılaştırması için AYNİ oturumdan iki model eğitilir:

- (A) SADECE ORTA (0°) anten -> 23 özellik (eski tek-anten hatıyla birebir)
- (B) 3 ANTEN BİRLEŞİK + çapraz-kanal -> 75 özellik

İkisi de aynı satırlardan, aynı zaman dilimlerinden, aynı şema ile değerlendirilir -> '3 anten ne kadar katkı sağlıyor' sorusuna doğrudan cevap.

7.2 Sızıntı-önleyici doğrulama: GroupKFold + zaman-bazlı holdout

Neden GroupKFold? Ardışık CSI paketleri zaman içinde birbirine yüksek korelasyonludur (aynı an, aynı konum, aynı gürültü). Rastgele k-fold kullanılırsa eğitim/test setine BİRBİRİNE ÇOK BENZER paketler düşebilir -> yapay/yanıltıcı yüksek skor (veri sızıntısı).

make_groups(): her etiket için ardışık BLOCK_SIZE=200 paket aynı 'gruba' atanır; GroupKFold bu grupları asla bölmeden katlar oluşturur.

split_holdout(): her etiketin son %20'lik ZAMAN dilimi tamamen ayrılır, eğitime hiç sokulmaz — 'modelin daha önce hiç görmediği, farklı bir zaman diliminden gelen veride ne kadar başarılı?' sorusunun cevabıdır.

8.1 6 algoritmanın yarıştırılması ve 'şampiyon' seçimi

Her iki özellik seti (23 ve 75 boyutlu) için AYNI 6 algoritma yarıştırılır:

LogReg (L2) · Linear SVM · RBF SVM · Random Forest ·
Gradient Boosting · KNN-7

(SVM'ler CalibratedClassifierCV ile sanılır — ham SVM skorları olasılık DEĞİLDİR, kalibrasyon onları gerçekçi olasılığa çevirir; ağaç-tabanlı modeller zaten predict_proba ürettiği için sarılmaz.)

Her model GroupKFold ortalamasına (cv_mean) göre sıralanır; en yüksek cv_mean'e sahip model 'şampiyon' seçilip ayrı tutulan holdout setinde test edilir — bu, gerçek dünyada beklenen başarıya en yakın rakamdır.

8.2 Sonuç karşılaştırması (ilk eğitim turu — 69-boyutlu özellikle)

	GroupKFold CV	Holdout Test
Tek anten (ORTA)	91.1%	91.3%
3 anten birleşik	94.8%	91.2%
FARK (3anten - tek)	+3.7%	−0.1%

Confusion matrix'te zone_B -> zone_A yönünde ~%26 karışma gözlemlendi — bu, fiziksel olarak yakın iki bölgenin CSI imzalarının kısmen örtüştüğünü ve/veya kalibrasyonsuz ağaç-tabanlı modellerin aşırı-güven eğilimini gösteriyor (bu rapor sürecinde tespit edilip 75-boyutlu çapraz-kanal özellikleriyle giderilmeye çalışıldı — Bölüm 6.2).

Şampiyon model + her kanalın ölü-subcarrier listesi joblib ile core/wifi_radar_model_multi.pkl olarak kaydedilir — eğitim ve canlı tahmin böylece AYNI ön-işleme parametrelerini paylaşır.

9.1 Başlangıç — modeli ve canlı veri akışını ayağa kaldırma

- 1) wifi_radar_model_multi.pkl yüklenir: model + öğrenilmiş parametreler (her kanal için ölü-subcarrier listesi, sınıf etiketleri labels)
- 2) Portlar bulunur ve [SOL,ORTA,SAĞ] sırasına dizilir, ESP32 IP'leri keşfedilir, traffic booster başlatılır (Bölüm 2.1 ile birebir aynı)
- 3) ESP32MultiReader ile Bölüm 3'teki AYNI olay-tabanlı/kalibreli birleştirme çalışır -> her an için taze bir 192-boyutlu birleşik CSI vektörü üretilir ve on_combined() callback'ine iletilir

9.2 Yeni örnekten özellik çıkarımı — eğitimle BİREBİR AYNI yol

on_combined() her yeni birleşik örnekte çalışır:

```
rows = [arr[ch*64 : (ch+1)*64] for ch in range(3)]
x = build_multi_feature_vector(rows, channel_dead).reshape(1, -1)
proba = model.predict_proba(x)[0]
```

ÖNEMLİ: build_multi_feature_vector() ve channel_dead listesi BİREBİR Bölüm 6 ve 7'de eğitimde kullanılanlarla AYNIDIR. Yeni gelen örnek ESKİ örneklerle kıyaslanmaz — eğitimde öğrenilen 'ölü subcarrier' bilgisiyle aynı şekilde temizlenir, aynı şekilde normalize edilir, aynı 75-boyutlu uzaya izdüşürülür. Böylece model, daha önce HİÇ GÖRMEDİĞİ bu yeni vektörü, eğitimde öğrendiği karar sınırlarına göre değerlendirir.

10.1 Olasılığa dönüşüm — 'birebir eşleştirme' DEĞİL, sınıflandırma

predict_proba(x)[0] -> [P(empty), P(zone_A), P(zone_B)] gibi toplam 1.0 olan bir olasılık dağılımı döner. Bu, 'en yakın eğitim örneğini bul'
DEĞİL, 'öğrenilen istatistiksel modele göre bu vektörün her sınıfa ait OLMA OLASILIĞI nedir' sorusunun cevabıdır.

Titreşimi azaltmak için son SMOOTH_N=5 tahminin ortalaması alınır (np.mean(recent_probs, axis=0)); en yüksek olasılıklı sınıf '>>> TAHMİN <<<' olarak işaretlenip ASCII çubuklarla (prob_bar) terminale basılır — ekran \033[2J\033[H ANSI kodlarıyla TERM-bağımsız temizlenir.

✓ SORUNUN CEVABI — "A mı B mi yaptırıyoruz, evet öyle mi yapıyoruz?"

EVET — sistem tam tahmin ettiğiniz gibi çalışıyor, ama 'birebir eşleştirme' değil, bir SINIFLANDIRMA / GENELLEME süreci:

- Model, eğitimde gördüğü binlerce örnekten zone_A ve zone_B için GENELLENEBİLİR istatistiksel imzalar (75-boyutlu uzayda karar sınırları) öğrenmiştir — satırları 'ezberlememiştir'.
- Holdout testi (Bölüm 7.2) bunu kanıtlar: model, eğitimde HİÇ görmediği farklı bir zaman dilimindeki veride de ~%91 başarı gösteriyor — yani 'daha önce görülmemiş' veriyle çalışması NORMAL ve beklenen davranıştır.
- Canlı tahminde de aynı mantık geçerlidir: yeni CSI'nin önceki test serileriyle birebir uyuşmasına HİÇ GEREK YOKTUR — önemli olan, aynı ön-işleme/özellik-çıkarma hattından geçtikten sonra öğrenilen karar sınırının HANGİ TARAFINA düştüğüdür.